

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 2
по дисциплине «Анализ алгоритмов»

Тема Сравнительный анализ рекурсивного и нерекурсивного алгоритмов

Студент Атаев К.А.

Группа ИУ7-52Б

Преподаватели Волкова Л. Л., Строганов Д.В.

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

2 Конструкторская часть **6**

2.1 Нерекурсивный алгоритм 6

2.2 Рекурсивный алгоритм 7

2.3 Теоретическая оценка трудоёмкости и потребления памяти 8

2.3.1 Трудоёмкость нерекурсивного алгоритма 8

2.3.2 Трудоёмкость рекурсивного алгоритма 8

2.4 Вывод 9

3 Технологическая часть **11**

3.1 Средства реализации 11

3.2 Реализация алгоритмов 11

3.3 Функциональные тесты 11

4 Исследовательская часть **12**

4.1 Технические характеристики 12

4.2 Демонстрация работы программы 12

4.3 Время выполнения реализаций алгоритмов 12

4.4 Вывол 14

ЗАКЛЮЧЕНИЕ **15**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **16**

ПРИЛОЖЕНИЕ А **17**

ВВЕДЕНИЕ

В рамках данной лабораторной работы рассматривается задача вывода всех целых чисел из заданного диапазона $[A, B]$, где $A \leq B$, при условии, что рекурсивная реализация должна обходиться без использования циклов. Для достижения поставленной цели необходимо решить следующие задачи:

1) разработать и описать рекурсивный и нерекурсивный алгоритмы решения задачи;
2) реализовать оба алгоритма;
3) выполнить оценку трудоёмкости каждого алгоритма;
4) провести анализ объёма памяти, необходимого для выполнения каждой реализации;
5) сравнить временные затраты реализаций алгоритмов;

1 Аналитическая часть

В данном разделе рассматриваются рекурсивный и нерекурсивный подходы к решению задач, основанных на последовательной обработке числовых диапазонов. Рекурсивные алгоритмы реализуют принцип декомпозиции — разбиение задачи на подзадачи того же типа, что обеспечивает простоту и математическую изящность описания. Нерекурсивные (итеративные) алгоритмы, напротив, опираются на циклы и явное управление состоянием, что повышает эффективность и снижает затраты памяти.

Рекурсивный алгоритм основан на принципе декомпозиции задачи, где основная проблема разбивается на более простые подзадачи того же типа. В данном случае алгоритм последовательно выводит текущее число и вызывает себя для следующего числа, пока не достигнет конечного значения. Такой подход демонстрирует наглядность математического решения, но имеет ограничения по глубине рекурсии [1].

Нерекурсивный алгоритм использует итеративный подход с явным управлением состоянием через цикл. Этот метод характеризуется последовательным перебором всех чисел в заданном диапазоне с минимальными накладными расходами.

Ключевое различие:

— потребление памяти. Рекурсивный метод требует дополнительной памяти для хранения контекста вызовов в стеке, что может привести к его переполнению при больших диапазонах. Итеративный подход использует фиксированное количество памяти независимо от размера задачи [2].

Вывод

В аналитической части были рассмотрены рекурсивный и нерекурсивный алгоритмы, их принципы работы, особенности и различия в потреблении памяти.

2 Конструкторская часть

2.1 Нерекурсивный алгоритм

Схема алгоритма приведена на рисунке 2.1.

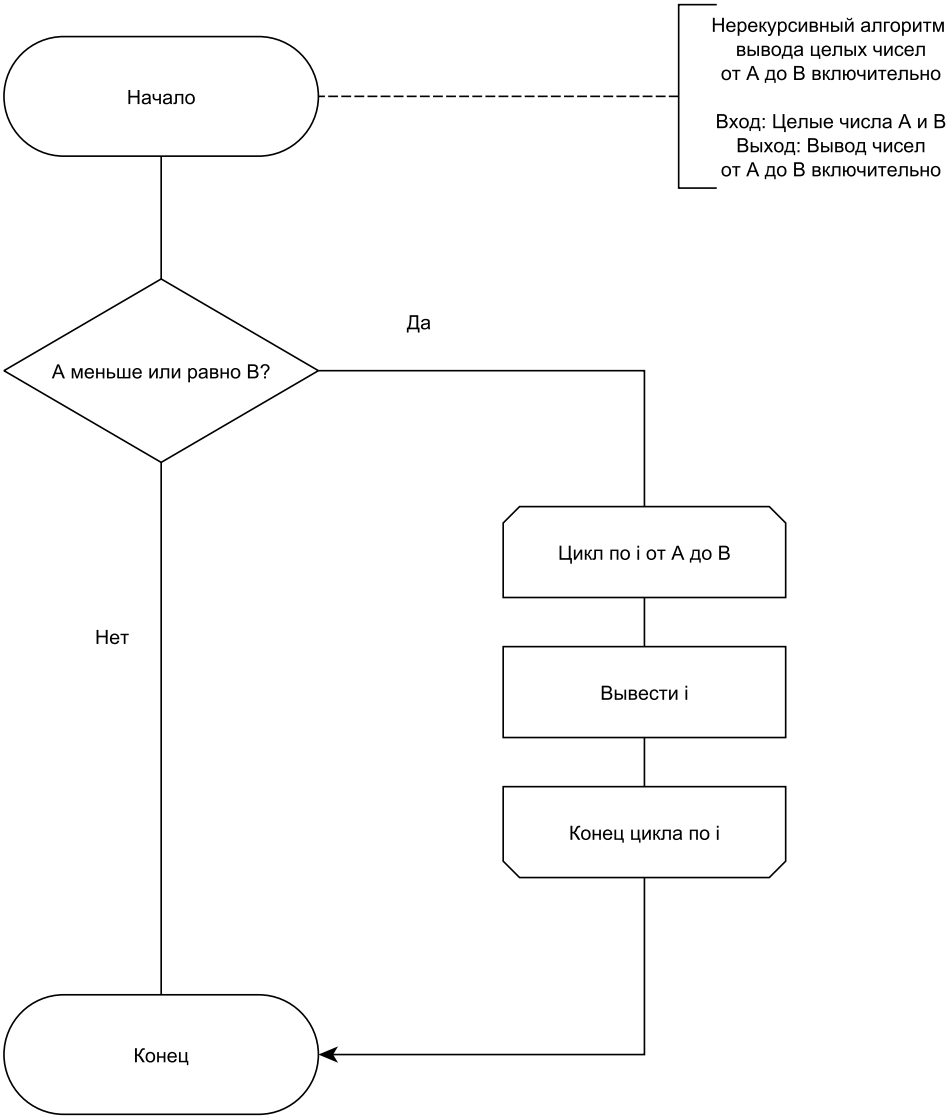


Рисунок 2.1 — Схема нерекурсивного алгоритма

2.2 Рекурсивный алгоритм

Схема алгоритма приведена на рисунке 2.2.

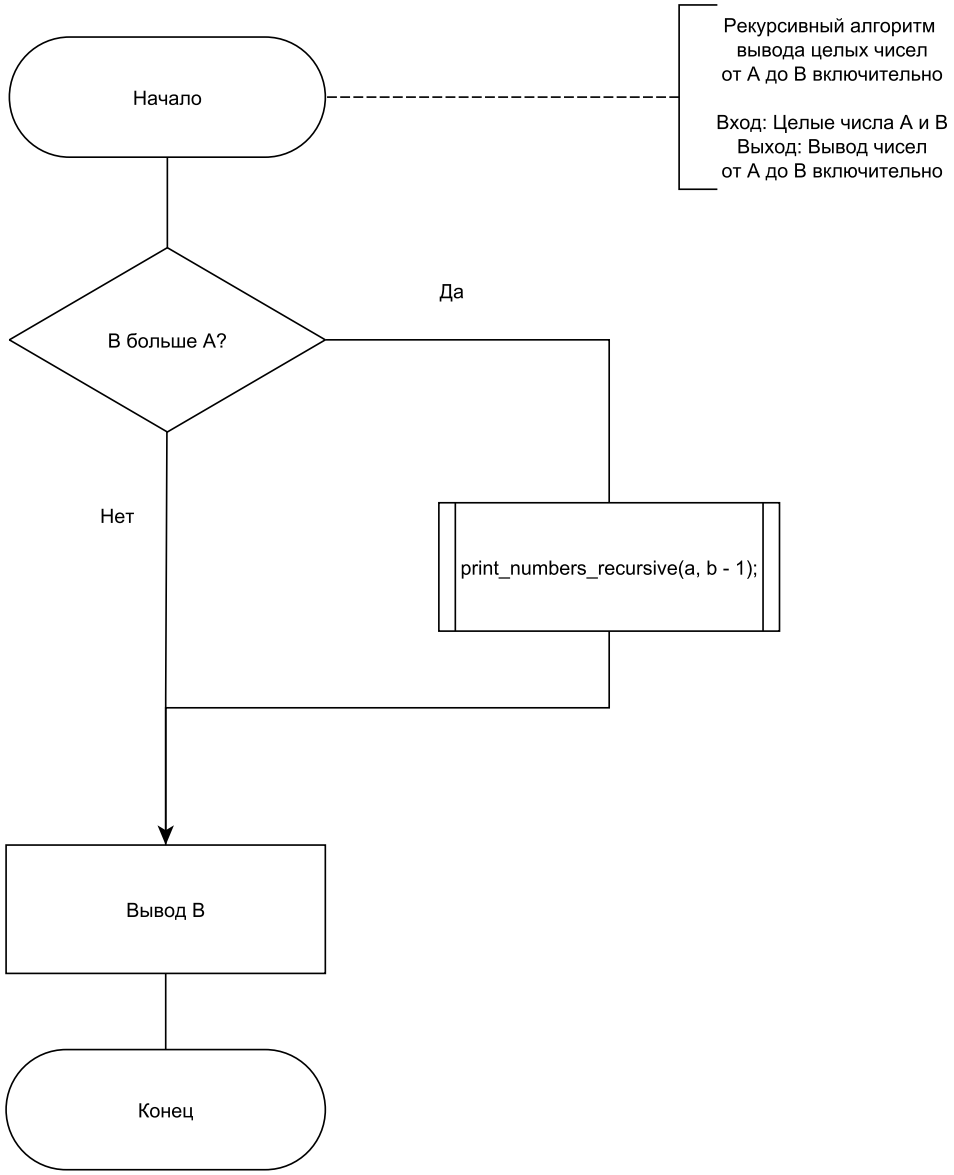


Рисунок 2.2 — Схема рекурсивного алгоритма

2.3 Теоретическая оценка трудоёмкости и потребления памяти

2.3.1 Трудоёмкость нерекурсивного алгоритма

Нерекурсивная реализация выполняет единственный вызов функции и цикл, проходящий по диапазону значений от a до b.
Так как функция вызывается один раз, накладные расходы имеют постоянное значение:

$$T_{\text{call}} = O(1) \tag{2.1}$$

В теле функции выполняется цикл длиной N, где

$$N = b - a + 1 \tag{2.2}$$

Каждая итерация включает служебные операции и операцию вывода. Пусть время одной итерации равно $(t_s + t_o)$, где t_s — стоимость служебных действий, а t_o — стоимость вывода числа. Тогда суммарная трудоёмкость цикла равна:

$$T_{\text{loop}}(N) = N \cdot (t_s + t_o) \tag{2.3}$$

Общая трудоёмкость нерекурсивного алгоритма складывается из затрат на вызов функции (2.1) и выполнения цикла (2.3):

$$T_{\text{nonrec}}(N) = T_{\text{call}} + T_{\text{loop}}(N) \tag{2.4}$$

Подставляя выражения (2.1) и (2.3), получаем:

$$T_{\text{nonrec}}(N) = O(1) + \Theta(N) = \Theta(N) \tag{2.5}$$

Таким образом, асимптотическая оценка времени выполнения нерекурсивного алгоритма равна $\Theta(N)$, где $N = b - a + 1$ (2.2).

2.3.2 Трудоёмкость рекурсивного алгоритма

Для анализа рекурсивного варианта введём параметры:

$$p = 2, \quad r = 3, \quad k = 0, \quad l = 0, \quad f = 1 \tag{2.6}$$

Память, выделяемая при каждом вызове функции, определяется как сумма параметров, локальных переменных, служебных регистров и одной дополнительной ячейки для хранения адреса возврата:

$$F = f + p + k + r + l + 1 \tag{2.7}$$

После подстановки значений из (2.6) имеем:

$$F = 1 + 2 + 0 + 3 + 0 + 1 = 7 \tag{2.8}$$

Глубина рекурсии обозначается через d и определяется числом последовательных вызовов:

$$d = b - a + 1 \tag{2.9}$$

Тогда общий объём используемой памяти вычисляется по формуле:

$$M_{\text{rec}} = F \cdot d \tag{2.10}$$

С учётом (2.8) и (2.9):

$$M_{\text{rec}} = 7 \cdot (b - a + 1) \tag{2.11}$$

Асимптотически:

$$M_{\text{rec}} = \Theta(b - a + 1) \tag{2.12}$$

Каждый рекурсивный вызов выполняет проверку условия, арифметические операции и вывод значения. Пусть:

$$f_{\text{leaf}} = 1 + 2 = 3, \tag{2.13}$$

$$f_{\text{internal}} = 1 + 1 + 2 = 4, \tag{2.14}$$

$$f_{\text{overhead}} = 2 \cdot F = 14 \tag{2.15}$$

Тогда общее время работы алгоритма:

$$T_{\text{rec}} = d \cdot f_{\text{overhead}} + (d - 1) \cdot f_{\text{internal}} + f_{\text{leaf}} \tag{2.16}$$

После упрощения (2.16) получаем:

$$T_{\text{rec}}(d) = 18d - 1 \tag{2.17}$$

Подставляя значение глубины рекурсии из (2.9):

$$T_{\text{rec}}(b, a) = 18 \cdot (b - a + 1) - 1 = \Theta(b - a + 1) \tag{2.18}$$

2.4 Вывод

Рекурсивный алгоритм имеет линейную временную сложность $T_{\text{rec}} = \Theta(b - a + 1)$ (см. 2.18) и линейную оценку потребляемой памяти $M_{\text{rec}} = \Theta(b - a + 1)$ (см. 2.12). Таким образом, нерекурсивная реализация является более эффективной как по времени выполнения,

так и по потреблению памяти.

3 Технологическая часть

В данном разделе приведены средства реализации, реализации алгоритмов и функциональные тесты

3.1 Средства реализации

Для реализации был выбран язык C++ в качестве среды разработки использовалась Visual Studio Code с компилятором g++. Для модульного тестирования применялась библиотека Check, а для визуализации результатов —утилита gnuplot.

3.2 Реализация алгоритмов

В этом разделе представлены реализации алгоритмов рекурсивного и нерекурсивного вывода чисел от A до B включительно.

В листингах А.1 —А.2 представлены реализации алгоритмов.

3.3 Функциональные тесты

Для проверки корректности реализации алгоритмов умножения матриц были разработаны функциональные тесты, охватывающие различные классы эквивалентности. Результаты тестирования представлены в таблице **??**.

Вывод

В технологической части были представлены листинги реализаций рекурсивного и итеративного алгоритмов вывода чисел в диапазоне, были описаны выбранные средства разработки и приведены функциональные тесты. Тестирование подтвердило корректность работы обоих алгоритмов для различных классов входных данных.

4 Исследовательская часть

В данном разделе будут приведены примеры работы программа, а также проведен сравнительный анализ алгоритмов при различных ситуациях на основе полученных данных.

4.1 Технические характеристики

Технические характеристики устройства, на котором выполнялось тестирование:

— операционная система: Windows 11 домашняя [6];

— память: 16 Гбайт;

— процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50ГГц, 2496 МГц, ядер: 4. [5]

4.2 Демонстрация работы программы

На рисунке 4.1 приведён пример работы программы.

```
-------------- Menu --------------
1 - Рекурсивный вывод чисел
2 - Итеративный вывод чисел
3 - Оба алгоритма
4 - Замерить время
0 - Выход
Выбор: 3
Введите два целых числа A и B (A ≤ B): 1 10
Рекурсивный вывод: 1 2 3 4 5 6 7 8 9 10
Итеративный вывод: 1 2 3 4 5 6 7 8 9 10
```

Рисунок 4.1 — Пример работы программы

4.3 Время выполнения реализаций алгоритмов

В таблице 4.1 приведены результаты замеров процессорного времени выполнения реализаций алгоритмов рекурсивного и нерекурсивного вывода чисел в диапазоне от A до B

Таблица 4.1 — Результаты замеров времени для рекурсивного и итеративного алгоритмов

Количество выводимых чисел	**Рекурсивный, мс**	**Итеративный, мс**
100	0.022	0.019
300	0.020	0.017
500	0.024	0.021
700	0.033	0.029
900	0.045	0.038
1100	0.054	0.047
1300	0.061	0.054
1500	0.069	0.060
1700	0.082	0.071
1900	0.090	0.078
2100	0.098	0.084
2300	0.110	0.096
2500	0.119	0.104
2700	0.127	0.112
2900	0.139	0.119
3100	0.146	0.127
3300	0.155	0.135
3500	0.167	0.143
3700	0.171	0.150
3900	0.194	0.171
4100	0.195	0.171
4300	0.202	0.178
4500	0.214	0.187
4700	0.234	0.206
4900	0.241	0.208
5100	0.250	0.218
5300	0.284	0.247
5500	0.289	0.251
5700	0.275	0.240
5900	0.289	0.250
6000	0.290	0.253

По таблице 4.1 был построен график представленный на рисунке 4.2.

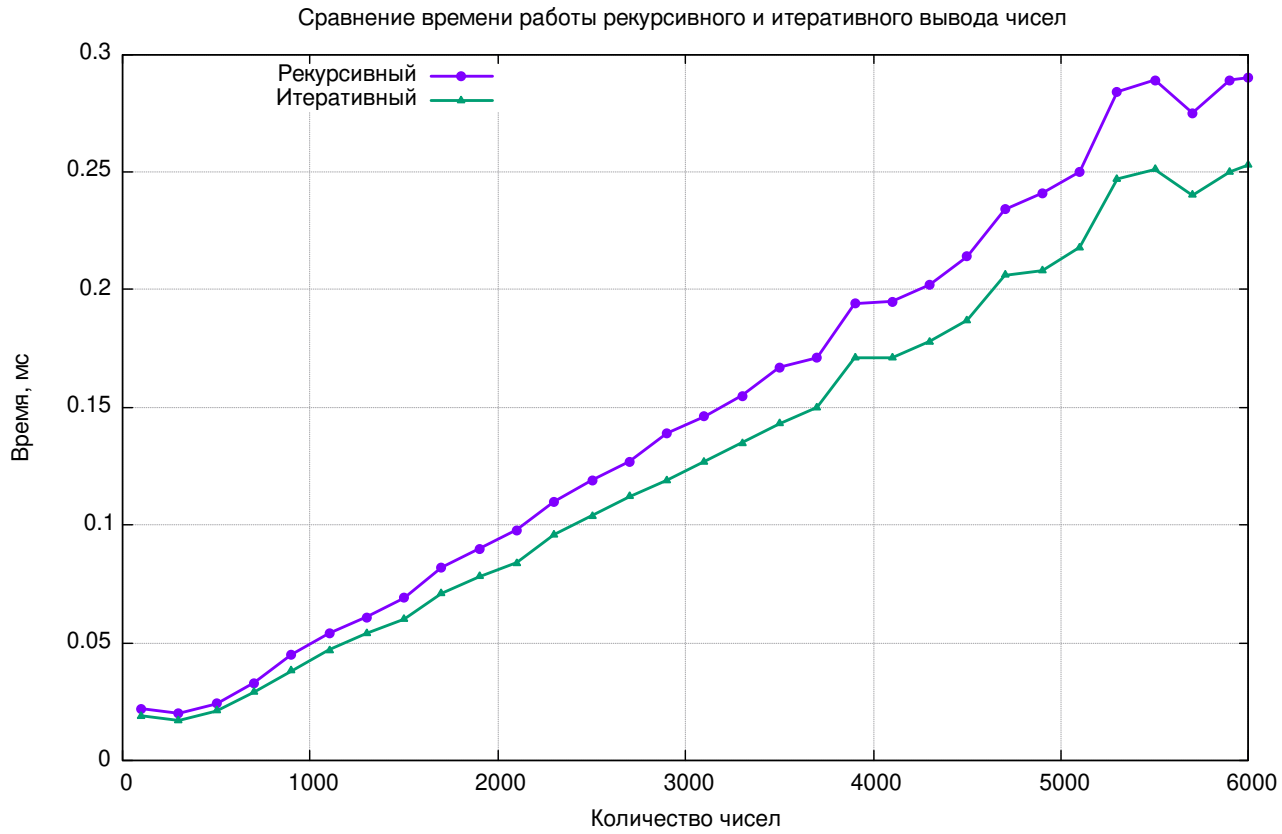


Рисунок 4.2 — График времени выполнения алгоритмов

4.4 Вывол

Были выполнены замеры процессорного времени для рекурсивной и итеративной реализаций алгоритмов вывода чисел в диапазоне. Анализ полученных данных показал, что итеративный алгоритм демонстрирует стабильное преимущество по скорости выполнения над рекурсивным подходом.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были реализованны и исследованы алгоритмы рекурсивного и нерекурсивного вывода чисел в диапазоне. Все цели лабораторной работы были достигнуты, а задачи выполнены.

1) разработаны и описаны рекурсивный и нерекурсивный алгоритмы решения задачи;
2) реализованы оба алгоритма;
3) выполнена оценка трудоёмкости каждого алгоритма;
4) реализован каждый алгоритм;
5) проведен анализ объёма памяти, необходимого для выполнения каждой реализации;
6) проведено сравнение временных затрат реализаций алгоритмов;

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Головешкин В. А., Ульянов А. В. Теория рекурсии для программистов. — М.: Наука, 2020. Режим доступа: https://example.com/recursion-theory (дата обращения: 05.11.2025)

2. Валединский В. Д., Корнев А. А. Методы программирования в задачах и примерах. — СПб.: Питер, 2018. Режим доступа: https://example.com/programming-methods (дата обращения: 05.11.2025)

3. Working Draft, Standard for Programming Language C++ [Электронный ресурс] // Open-std : [сайт]. — Режим доступа: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf (дата обращения: 5.11.2025)

4. GNU Compiler Collection (GCC) documentation [Электронный ресурс] // GCC : [сайт]. — Режим доступа: https://gcc.gnu.org/onlinedocs/ (дата обращения: 6.11.2025)

5. Intel® Core™ i5-10300H Processor [Электронный ресурс] // Intel : [сайт]. — Режим доступа: https://ark.intel.com/content/www/us/en/ark/products/201837/intel-core-i5-10300h-processor-8m-cache-up-to-4-50-ghz.html (дата обращения: 7.11.2025)

6. Windows 11 Home [Электронный ресурс] // Microsoft : [сайт]. — Режим доступа: https://www.microsoft.com/ru-ru/windows/windows-11 (дата обращения: 8.11.2025)

ПРИЛОЖЕНИЕ А

```
void print_numbers_recursive(int a, int b)
{
  if (b > a)
  {
    print_numbers_recursive(a, b - 1);
  }

  std::cout << b;
}
```

Листинг А.1 — Рекурсивная функция вывода чисел в диапазоне от A до B

```
void print_numbers_iterative(int a, int b)
{
  if (a <= b)
  {
    for (int i = a; i <= b; i++)
    {
      std::cout << i;
      if (i < b)
      {
        std::cout << " ";
      }
    }
  }
}
```

Листинг А.2 — Итеративная функция вывода чисел в диапазоне от A до B